

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	31 January 2026
Team ID	LTVIP2026TMIDS81120
Project Name	Smart Sorting: Transfer Learning for Identifying Rotten Fruits and Vegetables
Maximum Marks	4 Marks

Install Required Libraries

Install the necessary Python packages to perform image processing, deep learning model training, and web application development.

Libraries Used:

- NumPy
- Pandas
- Matplotlib
- TensorFlow / Keras
- Scikit-learn
- Flask
- Pillow / OpenCV

2 Data Collection

Collect labeled fruit image datasets representing fresh and spoiled categories. The dataset should be organized into structured folders for training and testing.

Tools/Libraries Used:

- Image datasets (local folders / public datasets)
 - Pandas (for metadata handling, if required)
 - File system management
-

3 Data Preprocessing

▪ Import the Libraries

Load required Python libraries to handle image processing, visualization, and machine learning tasks.

Libraries: NumPy, Pandas, Matplotlib, TensorFlow/Keras

▪ Importing the Dataset

Load images from directory folders using image generators or dataset loaders.

Libraries: TensorFlow/Keras

▪ Understanding Dataset Structure

Verify folder organization, class labels, and image counts to ensure balanced training.

Libraries: Python OS utilities, Pandas

▪ Image Preprocessing

Resize, normalize, and augment images to improve model learning and generalization.

Libraries: TensorFlow/Keras, Pillow

▪ Data Visualization

Visualize sample images and training metrics to understand patterns and model behavior.

Libraries: Matplotlib

▪ Splitting Training & Validation Data

Prepare training and validation sets for performance evaluation.

Libraries: TensorFlow/Keras

Model Building

▪ Training the Model

Train a convolutional neural network to learn visual patterns distinguishing fresh and spoiled fruits.

Libraries: TensorFlow/Keras

▪ Evaluation of Model

Measure model performance using accuracy and validation metrics.

Libraries: TensorFlow/Keras, Scikit-learn

▪ Saving the Model

Store the trained model for integration into the web application.

Libraries: TensorFlow/Keras, Joblib

5 Application Building

▪ Create an HTML Interface

Design a web page allowing users to upload fruit images.

Technologies: HTML, CSS

▪ Backend Development

Develop Flask backend logic to load the trained model, process uploaded images, and return predictions.

Libraries: Flask, TensorFlow/Keras, NumPy

6 Final UI – Web Application Dashboard

Create a user-friendly interface that displays uploaded images and freshness prediction results clearly.

Technologies/Libraries:

- Flask
- HTML
- CSS
- Bootstrap